

Algoritmo Guloso com Estratégia de Ocupação de Cantos e Busca Local para Minimização de Sobras no Corte de Chapas Metálicas.

Luiz Eduardo Albano Giovanetti¹, Marco Antonio De Castro Barbosa²

¹Engenharia de Computação - Universidade Tecnológica Federal do Paraná – UTFPR.

²Departamento Acadêmico De Informática - Universidade Tecnológica Federal do Paraná – UTFPR.

giovanetti@alunos.utfpr.edu.br, mbarbosa@utfpr.edu.br

Abstract:

The Rectangle Packing Problem consists of placing rectangular pieces on a sheet so as to minimize wasted area. Since this problem is classified as NP-hard, exact solutions are unfeasible for large-scale instances, making heuristic approaches a practical alternative. This work proposes and evaluates a greedy algorithm based on the Corner-Occupying Action strategy of Chen and Huang (2007), complemented by two local search heuristics for cutting optimization. The algorithms were implemented in the C programming language and evaluated benchmark instances from the literature and instances created for this work. Results show that the local search extensions significantly reduce trim loss compared to the base greedy algorithm, highlighting a clear trade-off between solution quality and execution time between the two approaches.

Keywords: *Greedy Algorithm, Waste Minimization, NP-Hard, Corner Occupying Strategy, Cutting Problem.*

Resumo:

O Problema de Corte de Retângulos consiste em posicionar peças retangulares em uma chapa de modo a minimizar a área desperdiçada. Por ser classificado como NP-difícil, abordagens exatas são inviáveis para instâncias de grande porte, tornando as heurísticas uma alternativa prática. Este trabalho propõe e avalia um algoritmo guloso baseado na estratégia de Ocupação de Cantos de Chen e Huang (2007), complementado por duas heurísticas de busca local para otimização do aproveitamento da chapa. Os algoritmos foram implementados em linguagem C e avaliados sobre instâncias de referência da literatura e instâncias *benchmark* criadas para este trabalho. Os resultados demonstram que as extensões por busca local reduzem significativamente a taxa de sobra em relação ao guloso base, com destaque para o compromisso entre qualidade de solução e tempo de execução observado entre as duas abordagens.

Palavras-chave: Algoritmo Guloso, Minimização de desperdício, NP-Difícil, Ocupação de Cantos, Problema de Corte.

1. Introdução

O Problema de Corte de Retângulos (*Rectangle Packing Problem*, RPP) consiste em posicionar diversos conjuntos de peças retangulares, nem sempre idênticas, dentro de um espaço retangular maior, de modo a maximizar a área utilizada [Scheithauer; Sommerweis, 1998]. Esse problema consiste, essencialmente, em encontrar um padrão de empacotamento que minimize a área não ocupada.

Seguindo a tipologia de Dyckhoff (1990), o RPP é classificado como um problema do tipo 2/B/O/F, ou seja, bidimensional, com empacotamento de subconjuntos de peças em uma única região maior e com tipos distintos de peças. A área não utilizada resultante é denominada *trim loss*, que representa a perda de corte, sendo definida como a porção da chapa que não foi aproveitada.

Esse problema possui aplicações diretas em indústrias de corte de chapas metálicas, compensados, vidros e tecidos, além de áreas como layout de circuitos integrados e diagramação editorial [Gava; *et al.*, 2025].

A motivação deste trabalho é avaliar a eficácia de uma abordagem gulosa para o RPP em cenários práticos de corte de chapas metálicas. Tal motivação se justifica pelo custo significativo associado ao desperdício de material em ambientes industriais, uma vez que mesmo uma redução de 1% na taxa de sobra pode representar economias expressivas em produção em larga escala. Esse aspecto é reforçado por Gava, *et al.* (2025), que destacam que o problema de corte pertence à classe NP-difícil dos Problemas de Corte e Empacotamento (PCE), além de evidenciar que o desperdício de matéria-prima em indústrias metalúrgicas eleva os custos de produção e gera impactos ambientais relevantes.

Com base nesse contexto, o presente trabalho tem como objetivo implementar e comparar três abordagens heurísticas para o Problema de Corte de Retângulos aplicado ao corte de chapas metálicas. A primeira contribuição é a reimplementação do algoritmo guloso de Ocupação de Cantos (*Corner-Occupying Action*) proposto por Chen e Huang (2007). A segunda é uma extensão por busca local que explora permutações na ordem de prioridade dos tipos de peças, buscando ordens que reduzam o desperdício. A terceira é uma extensão de busca local que avalia a rotação de peças não quadradas, testando se a orientação alternada de cada tipo melhora o aproveitamento da chapa.

2. Referencial Teórico

2.1 Definição Formal do Problema

Dado uma chapa retangular R_0 , com largura W_0 , e altura H_0 , e um conjunto de n peças retangulares $R_1, R_2, \dots, R_n$, com dimensões $w_i \times h_i$, o Problema RPP consiste em posicionar o maior número possível de peças dentro da chapa, respeitando as seguintes restrições:

- i. As bordas de cada peça devem ser paralelas às bordas da chapa, não sendo permitida rotação livre;

- ii. Nenhum par de peças pode se sobrepor.

O objetivo é minimizar a taxa de área não utilizada (*Trim Loss*), definida como:

$$\text{Sobra (\%)} = ((\text{Área total} - \text{Área utilizada}) / \text{Área total}) * 100$$

O RPP é classificado como um problema NP-difícil por redução ao problema de empacotamento em caixas (*Bin Packing*) [Garey; Johnson, 1979]. Isso implica que não se conhece um algoritmo de tempo polinomial capaz de garantir a solução ótima para todas as instâncias, tornando necessárias abordagens heurísticas e aproximativas na prática.

2.2 Algoritmos Gulosos

Um algoritmo guloso (*greedy algorithm*) constrói a solução de forma incremental, tomando, a cada etapa, a decisão localmente ótima, sem revisitar escolhas anteriores [Cormen; *et al.*, 2009]. Embora essa estratégia não garanta a obtenção do ótimo global, frequentemente produz soluções de alta qualidade com custo computacional polinomial.

A complexidade do algoritmo base implementado é $O(I \times T \times K)$, em que I representa o número de iterações (limitado à quantidade de peças posicionadas), T corresponde ao número de tipos de corte e K ao número de cantos candidatos ativos. Na prática, o tempo de execução é extremamente reduzido, mesmo para instâncias com centenas de tipos, o que justifica a adoção dessa abordagem em ambientes industriais com restrições de tempo real.

2.3 Estratégia de Ocupação de Cantos

A estratégia central do algoritmo proposto por Chen e Huang (2007) é a Ação de Ocupar Cantos (*Corner-Occupying Action*), segundo a qual toda peça posicionada deve ocupar um “canto” formado pela interseção das bordas de peças previamente inseridas ou das paredes da chapa.

Formalmente, um canto é definido como um ponto candidato (x, y) , gerado sempre que uma nova peça é inserida. A cada inserção, surgem dois novos cantos: um na posição superior esquerda $(x, y + h)$ e outro na posição inferior direita $(x + w, y)$ do retângulo posicionado. A cada iteração, o algoritmo avalia todos os pares viáveis do tipo (canto, tipo de peça) e seleciona aquele que apresenta o maior grau de ocupação.

O grau de ocupação de um candidato é determinado pela quantidade de bordas do novo retângulo que estão em contato com as paredes do contêiner ou com as laterais de peças já posicionadas. Quanto maior esse grau, melhor o encaixe da peça, resultando na redução de espaços ociosos.

2.4 Busca Local

A Busca Local (*Local Search*) é um paradigma de otimização que parte de uma solução inicial e a melhora iterativamente por meio da exploração de sua vizinhança [Aarts; Lenstra, 1997]. Uma solução vizinha é obtida pela aplicação de um pequeno movimento sobre a solução corrente, uma perturbação controlada que gera uma configuração alternativa dentro do mesmo espaço de soluções. O processo se repete enquanto existir algum vizinho que melhore a solução atual, encerrando quando nenhum movimento produz ganho, condição denominada ótimo local.

No contexto deste trabalho, a busca local é aplicada como uma etapa de pós-processamento do algoritmo guloso. Dado que ele constrói sua solução de forma incremental e sem revisitar decisões anteriores, é natural que a ordem de prioridade dos tipos de peças e a orientação em que cada peça é inserida influenciem diretamente o aproveitamento final da chapa. A busca local atua justamente sobre esses dois graus de liberdade, explorando configurações alternativas que o guloso não examina.

3. Metodologia

O desenvolvimento deste trabalho envolve a implementação de um algoritmo guloso para o problema de corte de chapas metálicas, bem como a criação de ferramentas auxiliares para leitura de instâncias e visualização dos resultados. O código-fonte completo da solução, incluindo a implementação em linguagem *C* e o *script* de visualização em *Python*, está disponível em repositório público para fins de reprodutibilidade [Giovanetti, 2026].

3.1 Visão Geral da Implementação

O algoritmo foi implementado na linguagem *C*, seguindo o padrão *C99*, sendo organizado em um único arquivo-fonte. A implementação utiliza alocação dinâmica de memória e apresenta uma separação clara entre as estruturas de dados, a lógica do algoritmo e os módulos de entrada e saída.

As principais estruturas de dados utilizadas são:

- **Retângulo:** armazena a posição (x,y), as dimensões e o tipo de cada peça posicionada;
- **Canto:** estrutura responsável por manter um conjunto dinâmico de pontos candidatos para inserção de novas peças;
- **TipoCorte:** armazena as dimensões e a quantidade máxima permitida de cada tipo de peça;
- **Resultado:** armazena métricas associadas à execução, como área utilizada, área de sobra e taxa de ocupação da chapa.
- **Instância:** encapsula as informações da chapa, os tipos de corte, a ordem de prioridade e os resultados associados a cada caso de teste.

3.2 Leitura de Instâncias por Arquivo

As instâncias são lidas a partir de um arquivo de texto estruturado por meio de marcadores iniciados com o caractere “@”, conforme o modelo apresentado no arquivo de entrada.

O parser implementado realiza duas passagens sobre o arquivo: na primeira, são identificados os separadores de instâncias (“---”), permitindo a alocação adequada de memória; na segunda, os dados são efetivamente lidos e armazenados nas estruturas correspondentes.

A leitura das linhas é realizada com alocação dinâmica de memória, iniciando com um *buffer* de tamanho fixo e sendo redimensionado progressivamente por meio da função *realloc*. Essa abordagem permite lidar com linhas de tamanho arbitrário, como aquelas associadas à definição de prioridades (@PRIORIDADE), evitando problemas como estouro de *buffer*.

3.3 Algoritmo Guloso Base

O fluxo principal do algoritmo está implementado na função *executar_algoritmo_guloso*, sendo composto pelas seguintes etapas:

- **Inicialização:** o canto inicial (0,0) é inserido na lista de candidatos;
- **Avaliação:** para cada par formado por um canto candidato e um tipo de peça disponível, verifica-se a viabilidade da inserção (respeitando os limites da chapa e a ausência de sobreposição), além do cálculo do grau de ocupação;
- **Seleção gulosa:** é escolhido o par (canto, tipo de peça) que apresenta o maior grau de ocupação;
- **Atualização:** a peça é inserida na solução, o canto utilizado é removido e dois novos cantos são adicionados à lista de candidatos;
- **Término:** o processo é repetido até que não existam mais cantos capazes de acomodar qualquer tipo de peça disponível.

Essa estratégia segue diretamente o princípio da heurística gulosa, priorizando decisões locais que maximizam o aproveitamento imediato do espaço.

3.4 Busca Local por Permutação de Prioridades

Sabendo que o algoritmo guloso base é sensível à ordem de prioridade dos tipos de peças, a mesma chapa com a mesma coleção de peças pode ter taxas de sobra distintas dependendo da sequência em que os tipos são considerados. A busca Local por permutação explora essa sensibilidade de forma sistemática.

A heurística de Busca Local por Permutação de Prioridades parte da solução produzida pelo algoritmo guloso e define como vizinhança todas as soluções obtidas pela troca de posição de dois tipos de peças no vetor de prioridades, operador denominado *swap* na literatura de busca local [Aarts; Lenstra, 1997]. Dado que o

resultado do guloso depende da ordem em que os tipos são considerados, diferentes permutações podem produzir *layouts* com aproveitamentos distintos.

A busca opera da seguinte forma, para cada par (i, j) de tipos com quantidade irrestrita ($max_quantidade = 0$), realiza a troca de suas posições no vetor de prioridades e executa novamente o algoritmo guloso. Caso a nova solução apresente menor área de sobra, a troca é mantida e o processo recomeça do início do vetor (estratégia *first improvement*), caso contrário, a troca é desfeita. O algoritmo converge quando nenhuma troca de par produz redução de sobra, caracterizando um ótimo local na vizinhança de permutação.

A escolha de considerar apenas os tipos de peças com quantidade irrestrita fundamenta no fato de que os tipos sujeitos a uma quantidade máxima previamente definida já possuem seu aproveitamento naturalmente limitado por essa restrição de disponibilidade. Dessa forma, alterações em sua posição no vetor de prioridades tendem a exercer influência reduzida sobre a solução final obtida.

A complexidade adicional introduzida pela Busca Local por Permutação de Prioridades é $O(T^2 \times C_G)$ por iteração do laço externo, onde T é o número de tipos sem restrição e C_G é o custo de uma execução do algoritmo guloso.

3.5 Busca Local por Rotação de Peças

A heurística de Busca Local por Rotação de Peças explora o fato de que peças retangulares não quadradas possuem duas orientações possíveis, a original (largura x altura) e a rotacionada (altura x largura), o problema original de Chen e Huang (2007) não considera rotação. Enquanto o algoritmo guloso base e a busca por permutação fixam as dimensões originais de cada tipo, esta heurística permite a troca de orientação, ampliando o espaço de solução explorado.

A cada iteração, percorre o conjunto de tipos de peças. Para cada i não quadrado ($largura \neq altura$), realiza a rotação de suas dimensões e executa o algoritmo guloso com a configuração modificada. Se a solução resultante apresentar menor área de sobra, a rotação é aceita e a varredura recomeça, caso contrário, as dimensões originais são restauradas. O processo termina quando nenhuma rotação individual produz redução de sobra.

Em relação à heurística de permutação e a de prioridades, a busca por rotação apresenta vizinhança de menor cardinalidade, o que implica menor custo computacional por iteração. Por explorarem dimensões distintas do espaço de soluções, as duas heurísticas são complementares.

A complexidade introduzida pela Busca Local por Rotação de Peças é $O(T \times C_G)$ por iteração do laço externo.

3.6 Integração com Python para Visualização

Após a execução do algoritmo em *C*, os resultados são exportados para um arquivo de saída contendo as coordenadas e dimensões de cada peça posicionada bem como as informações já calculadas durante a execução, como a taxa de ocupação e a área de sobra.

Um script em *Python* foi desenvolvido para leitura e visualização desses dados (*LerCortes.py*), utilizando a biblioteca *Matplotlib* para gerar representações gráficas dos *layouts* de corte. Nesse processo, cada peça é desenhada como um retângulo com bordas destacadas e preenchimento em tons de cinza, permitindo melhor distinção visual entre os elementos.

Além da representação gráfica, o *script* organiza e exibe as informações provenientes do arquivo de saída, como dimensões da chapa, quantidade de peças posicionadas e percentual de sobra. Para cada instância processada, é gerada uma imagem no formato *PNG*, possibilitando a análise visual dos resultados obtidos.

3.7 Geração das Instâncias

As instâncias utilizadas nos experimentos foram divididas em dois grupos. O primeiro grupo é composto pelas três instâncias originais do artigo de Chen e Huang (2007) - C11, C21 e C41, empregadas para fins de validação e comparação direta com os resultados reportados pelos autores. O segundo grupo é composto por treze instâncias *benchmark* próprias, denominadas B01 a B13, criadas para avaliar o comportamento dos algoritmos em cenários de escala e complexidade crescentes.

As instâncias B01 a B10 seguem uma progressão sistemática, o tamanho da chapa cresce de 150x150 até 600x600, em incrementos de 50 por lado, enquanto o número de tipos de peças cresce de 15 a 60, em incrementos de 5 tipos por instância. Cada instância combina peças quadradas, retângulos horizontais, retângulos verticais, peças pequenas, médias e grandes, de modo a exercitar diferentes aspectos do algoritmo de posicionamento. As três últimas instâncias, B11 (2500x2500, 100 tipos), B12 (4500x4500, 150 tipos) e B13 (6000x6000, 150 tipos), representam benchmarks de grande porte, projetados para avaliar o comportamento dos algoritmos sob alta demanda computacional.

A geração das instâncias foi realizada por meio de um *prompt* estruturado submetido ao modelo de linguagem *Claude* [Anthropic, 2026]. O *prompt* especificou o formato exato de entrada esperado pelo programa em *C*, as regras de dimensionamento de cada instância, a mistura de tipos de peças e uma semente pseudoaleatória fixa (*SEED* = 2026), garantindo que os dados sejam reproduzíveis por qualquer pessoa que submeta o mesmo *prompt* ao mesmo modelo. Essa abordagem permite que as instâncias sejam consideradas públicas e verificáveis, ainda que não pertençam a um repositório *benchmark* consolidado da literatura.

O *prompt* para a geração das instâncias encontra disponível no repositório *GitHub* do projeto, armazenado como arquivo *.txt*, permitindo que qualquer interessado reproduza exatamente as mesmas instâncias ao submeter ao modelo *Claude*.

Como as instâncias B01 a B12 foram geradas especificamente para este trabalho, não existem valores de referência na literatura para comparação direta. Dessa forma, a avaliação dessas instâncias é realizada de maneira relativa, os resultados dos três algoritmos são comparados entre si, tendo como métrica principal a taxa de sobra da chapa ao final do processo de corte. Já as instâncias C11, C21 e C41, provenientes do trabalho de Chen e Huang (2007), é possível realizar também uma comparação absoluta, confrontando os resultados obtidos com os valores reportados pelos autores originais.

3.8 Ambiente Computacional

Todos os experimentos foram executados em um computador com processador *Intel Core i5-3470* (3,20 GHz) e 16 GB de memória *RAM*, sob o sistema operacional *Windows 10*. Os algoritmos foram implementados em linguagem *C* e compilados com o compilador padrão do ambiente *Windows*. Para fins de comparação, Chen e Huang (2007) realizaram seus experimentos em um *IBM ThinkPad R40* equipado com processador *Pentium 4* a 2,0 GHz e 256 MB de memória *RAM*. A diferença entre as duas máquinas é expressiva, o processador utilizado neste trabalho opera com frequência 60% superior e conta com memória *RAM* aproximadamente 64 vezes maior, o que deve ser considerado ao interpretar as diferenças de tempo de execução reportadas entre os dois trabalhos. Ainda assim, a comparação permanece válida do ponto de vista qualitativo, uma vez que o foco da análise recai sobre a taxa de sobra e sobre a tendência de crescimento do tempo do tamanho das instâncias.

4. Resultados e Discussão

4.1 Comparação com o Artigo de Chen e Huang

A Tabela 1 apresenta os resultados obtidos pelos três algoritmos nas instâncias *C11*, *C21* e *C41*, originárias do trabalho de Chen e Huang (2007), permitindo comparação com os valores reportados pelos autores.

Tabela 1 - Comparação com Chen e Huang (2007) nas instâncias de referência.

Inst.	Chapa	Sobra% Ref.	Sobra % Guloso Base	Sobra % Exaustiva	Sobra % Rotação	Tempo Ref. (s)	Tempo Guloso Base (s)	Tempo Exaustiva (s)	Tempo Rotação (s)
C11	20x20	0,00	0,00	0,00	0,00	0,040	0,000	0,031	0,004
C21	40x15	0,00	0,00	0,00	0,00	0,510	0,000	0,000	0,002
C41	60x60	0,33	0,00	0,00	0,00	3,570	0,0260	31,57	1,086

Fonte: Autoria Própria.

As três implementações igualaram ou superaram os resultados do artigo original em todas as instâncias de referência. Na instância *C4I*, o algoritmo de Chen e Huang registrou sobra de 0,33%, enquanto os três algoritmos implementados nesse trabalho alcançaram aproveitamento total (0,00%). Esse resultado indica que a estratégia de Ocupação de Cantos, conforme reimplementada, é capaz de encontrar soluções de alta qualidade mesmo em instâncias com maior número de tipos de peças.

Em termos de tempo de execução, o algoritmo guloso base foi mais rápido que a referência em todas as instâncias, a diferença mais significativa ocorreu em *C4I*, onde o guloso levou 0,026 s contra 3,57 s do artigo original, diferença que pode ser atribuída à evolução do *hardware* e à eficiência da implementação em *C*. A busca local por rotação apresentou tempo intermediário em *C4I* (1,086 s), enquanto a busca local exaustiva por permutação levou 31,578 s nessa instância, evidenciando o custo adicional da exploração exaustiva da vizinhança de permutações.

4.2 Resultados das Instâncias Benchmark

A Tabela 2 apresenta os resultados comparativos dos três algoritmos nas treze instâncias *benchmark* *B01–B13*, organizadas em ordem crescente de tamanho de chapa.

Tabela 2 - Comparação dos três algoritmos nas instâncias *benchmark*.

Inst.	Chapa	Sobra % Guloso Base	Sobra % Exaustiva	Sobra % Rotação	Tempo Guloso Base (s)	Tempo Exaustiva (s)	Tempo Rotação (s)
B01	150x150	5,05	1,07	2,33	0,001	0,672	0,011
B02	200x200	9,52	2,27	8,97	0,000	0,063	0,003
B03	250x250	0,97	0,45	0,34	0,002	1,829	0,032
B04	300x300	2,14	0,30	2,03	0,011	3,093	0,146
B05	350x350	1,21	0,48	0,68	0,002	5,656	0,019
B06	400x400	1,30	0,45	1,15	0,006	6,797	0,081
B07	450x450	2,22	0,61	1,84	0,030	9,343	0,627
B08	500x500	3,70	0,97	2,16	0,007	18,20	0,116
B09	550x550	3,49	0,31	3,49	0,003	5,750	0,062
B10	600x600	2,34	0,03	1,74	0,005	5,624	0,057
B11	2500x2500	2,02	0,49	1,64	0,036	267,8	2,162
B12	4500x4500	2,78	0,12	1,44	0,070	96,62	2,757
B13	6000x6000	3,24	0,25	2,58	0,278	279,8	23,45

Fonte: Autoria Própria.

A busca local por permutação de prioridades (exaustiva) obteve os melhores resultados de aproveitamento em todas as treze instâncias *benchmark*, sendo a abordagem mais eficaz em termos de qualidade de solução.

A busca local por rotação de peças obteve desempenho intermediário: superou o guloso base em 11 das 13 instâncias, mas ficou abaixo da busca exaustiva em todas elas. Nas instâncias

B03 e *B09*, seu comportamento foi distinto: em *B03* alcançou sobra de 0,34%, levemente menor que a exaustiva (0,45%), sendo o único caso em que a rotação superou a permutação. Em *B09*, os resultados de rotação e guloso foram idênticos (3,49%), indicando que nessa instância nenhuma rotação individual de peça produziu melhora em relação à solução inicial do guloso.

A busca local exaustiva, por sua vez, apresentou crescimento expressivo do tempo nas instâncias de maior porte. Em *B11* (2500×2500 cm, 100 tipos) levou 267,803 s, e em *B13* (6000×6000 cm, 150 tipos) chegou a 279,866 s. Esse comportamento é coerente com sua complexidade $O(T^2 \times C_G)$ por iteração, com $T = 150$ tipos, cada um exigindo uma execução completa do guloso sobre a chapa.

A busca local por rotação apresentou tempos moderados em todas as instâncias, variando de 0,003 s (*B02*) a 23,454 s (*B13*), mantendo dentro de limites práticos mesmo nos *benchmarks* de grande porte. Sua complexidade adicional de $O(T \times C_G)$ por iteração justifica esse comportamento, a vizinhança é pequena e cada iteração exige apenas uma reexecução do guloso por tipo de peça não quadrado.

Vale destacar que o algoritmo guloso base, apesar de ser a abordagem mais simples, já apresentou bom desempenho em instâncias onde a mistura de peças se adapta naturalmente a chapa, como *B03* (0,97%) e *B05* (1,21%), evidenciando que a qualidade da solução inicial tem forte influência no resultado final das buscas locais.

O guloso base foi invariavelmente o mais rápido, com tempos inferiores a 0,3 s mesmo na maior instância (*B13*, 6000×6000 cm com 150 tipos). Esse comportamento é esperado, dado que sua complexidade é determinística e não envolve iterações de refinamento.

Existe um claro compromisso entre qualidade e tempo de execução, a busca exaustiva entrega as melhores soluções, mas a um custo computacional que pode ser proibitivo em cenários industriais com muitos tipos de peças e necessidade de resposta rápida. A busca por rotação representa um ponto de equilíbrio interessante, oferecendo melhoras consistentes de qualidade com tempo de execução muito inferior ao da abordagem exaustiva.

5. Conclusão

Este trabalho propôs e avaliou três abordagens heurísticas para o Problema de Corte de Retângulos aplicado ao corte de chapas metálicas, um algoritmo guloso baseado na estratégia de Ocupação de Cantos de Chen e Huang (2007), e duas extensões por busca local, sendo uma por exploração de permutações da ordem de prioridade dos tipos de peças e outra testando a rotação de peças não quadradas.

Os experimentos foram conduzidos sobre 16 instâncias no total: três instâncias de referência da literatura (*C11*, *C21* e *C41*) e treze instâncias *benchmark* próprias (*B01–B13*), com chapas variando de 150×150 a 6000×6000 e número de tipos entre 15 e 150. Os resultados permitiram as seguintes conclusões:

- O algoritmo guloso base demonstrou ser uma solução sólida e extremamente rápida, igualando ou superando os resultados de Chen e Huang (2007) em todas as instâncias de referência, com tempos de execução muito inferiores aos reportados no artigo original. Nas instâncias *benchmark*, produziu soluções com sobras entre 0,97% e 9,52%, evidenciando que a qualidade depende diretamente da compatibilidade entre o conjunto de peças e a geometria da chapa.
- A busca local por permutação de prioridades mostrou-se a abordagem de maior qualidade, produzindo as melhores soluções em 12 das 13 instâncias *benchmark*, com sobras frequentemente abaixo de 1%. Essa vantagem, contudo, vem acompanhada de custo computacional significativo, que cresce de forma quadrática com o número de tipos de peças, tornando a abordagem menos adequada para instâncias de grande porte quando o tempo de resposta é um fator crítico.
- A busca local por rotação de peças configurou-se como o melhor compromisso entre qualidade e eficiência: superou o guloso base em 11 das 13 instâncias e manteve tempos de execução muito menores que a abordagem exaustiva, mesmo nas instâncias de maior porte. Sua complexidade linear em relação ao número de tipos a torna escalável e aplicável em cenários industriais com restrições de tempo.

Referências

Anthropic. Claude (claude-sonnet-4-5). <https://claude.ai>, April.

Chen, D.; Huang, W. (2007). “Greedy algorithm for rectangle-packing problem. Computer Engineering”. <https://www.ecice06.com/EN/10.3969/j.issn.1000-3428.2007.04.055>, March.

Cormen, T. H. et al. (2009). Introduction to Algorithms, 3. ed. Cambridge: MIT Press.

Dyckhoff, H. (1990). “A typology of cutting and packing problems”. <https://www.sciencedirect.com/science/article/pii/037722179090350K>, April.

Garey, M. R.; Johnson, D. S. (1979). “Computers and Intractability: A Guide to the Theory of NP-Completeness”. New York: W. H. Freeman.

Gava, M. C. V. et al. (2025). “Otimização do corte de chapas na indústria metalúrgica usando algoritmo genético combinado com processamento de imagens digitais”, <https://periodicos.uninove.br/exacta/article/view/28772>, April.

Giovanetti, Luiz Eduardo Albano (2026). “Projeto-e-Analise-de-Algoritmo. GitHub, 2026”, <https://github.com/LuizGiovanetti/Projeto-e-Analise-de-Algoritmo/tree/main>, April.

Scheithauer, G.; Sommerweis, U. (1998). “4-block heuristic for the rectangle packing problem”. <https://www.sciencedirect.com/science/article/pii/S0377221796003591>, April.

Aarts, E.; Lenstra, J. K. (1997). Local Search in Combinatorial Optimization. Chichester: Wiley.